Kubernetes GPU Integration

for MLOps Workloads

A Modern Implementation Guide for

Production-Ready GPU Computing Infrastructure

June 22, 2026

Docker + Minikube + Kubernetes + GPU

Author

William Rodriguez

Líder de IA y Arquitecto de Soluciones

eCaptureDtech

Badajoz, Extremadura, España

Contact Information:

LinkedIn: es.linkedin.com/in/wisrovi-rodriguez

Email: william.rodriguez@ecapturedtech.com

GitHub: github.com/wisrovi

Expert in MLOps and Cloud Architecture

Contents

List of Figures

List of Tables

Listings

Chapter 1

Executive Summary

This technical manual outlines the comprehensive architectural mapping and infrastructure design of the *wisrovi SUITE*, a production-ready ecosystem for distributed computer vision, MLOps, and secure pipeline orchestration. Developed by William Rodriguez, AI Leader and Solutions Architect, this suite contains 26+ custom Python packages that bridge the gap between high-level architectural designs and resilient implementation.

The core of the ecosystem is governed by *wpipe*, a thread-safe pipeline execution engine featuring WAL-mode SQLite state storage, GIL bypass, and dynamic checkpoints. Database layers are modeled using type-safe Pydantic ORMs (*wsqlite*, *wredis*, *wclickhouse*), while security interfaces implement machine-locked credentials vaults (*wauth*) and ECDSA P-256 digital signatures (*wFabricSecurity*). The suite is validated in production by *NeuralForge AI*, a distributed training cluster coordinating YOLO and RT-DETR training workloads across multiple GPU-accelerated Docker nodes via Celery and Optuna.

This guide serves as a technical manual for deploying, configuring, and operating these high-performance systems, with detailed code examples, architectural diagrams, benchmarks, troubleshooting steps, and future integration paths with Generative AI (LangGraph, LangChain, RAG).

Chapter 2

Introduction

2.1 Background and Context

In modern enterprise software engineering, developing individual software scripts is straightforward, but building highly resilient, scalable, and secure production architectures remains a major challenge. Infrastructure tools must provide deterministic state transitions, handle connection pool locks, secure credentials, and utilize computational resources (such as GPUs) efficiently.

This document serves as a master blueprint for the *wisrovi SUITE*, a collection of 26+ libraries engineered in Python to modularize and secure enterprise applications. The project spans database wrappers, pipeline orchestrators, cryptographic secret managers, and MLOps adapters.

2.2 Author Profile and Professional Scope

William Rodriguez (wisrovi) is the AI Leader and Solutions Architect at eCaptureDtech, based in Badajoz, Extremadura, Spain. His professional background focuses on:

- **Generative AI & Agentic Systems**: Designing stateful multi-agent systems using LangGraph and LangChain, and advanced Retrieval-Augmented Generation (RAG) models.
- **Computer Vision**: Deep learning lifecycles for object detection using YOLO and RT-DETR.
- **High-Availability Python Engineering**: Constructing optimized, concurrent libraries leveraging Python's features.
- **DevOps & Cloud Architecture**: Containerization topologies (Docker, Kubernetes, Minikube), worker tasks scheduling (Celery, Redis), and Git CI/CD automated linting.

2.3 Document Goals and Workspace Scope

The primary objective of this manual is to document the entire workspace located at `/home/william.rodriguez/Documents/w_libraries`. This guide explains how each package operates, how the systems are integrated, how to install and run them, and how to verify their technical performance.

Chapter 3

The wisrovi SUITE Technical Architecture

3.1 Core Pipeline Orchestration (wpipe Ecosystem)

The orchestrator consists of five distinct sub-components under a unified layout:

1. **wpipe**: The core execution engine. Exposes sync (`Pipeline`) and async (`PipelineAsync`) runners, state persistence utilizing SQLite WAL mode, and GIL bypass via process pools.
2. **wpipe-steps**: Reusable step modules covering HTTP requests, database queries, and notifications.
3. **wpipe-plugins**: A community-supported registry for third-party extensions.
4. **wpipe-mcp**: A Model Context Protocol (MCP) server letting AI agents design pipelines.
5. **wpipe-api**: Process monitor REST API built with FastAPI to track worker node health.

3.2 Database ORMs and Caching

The database layer enforces strict Pydantic v2 schemas:

- **wsqlite**: A SQLite ORM with automatic table migrations (`TableSync`), connection sharing pools, and soft deletion mixins.
- **wredis**: Redis cache layer with expiring caching decorators and token-bucket rate limiters.
- **wclickhouse**: Fast bulk inserter mapped via clickhouse-connect.
- **Other ORMs**: Configured clients for MariaDB, MySQL, MongoDB, Elasticsearch, Databricks, and Snowflake.

3.3 Zero Trust Cryptography and Secret Vaults

* **wauth**: A machine-locked credentials manager. It reads system-level IDs (`/etc/machine-id`), hashes them with a local salt, and derives a unique 32-byte Fernet key. Secrets are stored in SQLite and can only be decrypted on the original host machine. * **wFabricSecurity**: Middlewares for Hyperledger Fabric providing ECDSA P-256 message signing, code integrity verification (SHA-256 checks), and token-bucket rate limiters.

3.4 MLOps and Multimedia

* **wyolo**: Manages YOLO and RT-DETR models. Orchestrates GPU topology checks, auto-sizes batches based on VRAM capacity, and streams logs to MLflow. * **ProcessAudio**: Data augmentation (stretching, noise injection) and feature extraction (MFCCs, mel-spectrograms) using librosa.

Chapter 4

Installation and Configuration Guide

4.1 Prerequisites and System Requirements

The suite requires a Linux environment with the following packages:

- Python 3.10 or higher.
- SQLite3 (configured with WAL support).
- Docker Engine and nvidia-container-toolkit (for GPU workloads).
- NVIDIA CUDA Driver (v12.0+ recommended).

4.2 Step-by-Step Installation

Each package under `w_libraries` is structured with a `setup.py` and can be installed locally:

```
# Clone or locate the repository directory
cd /home/william.rodriguez/Documents/w_libraries/w_libraries

# Install core engines
pip install -e wsqlite/
pip install -e wauth/
pip install -e wpipe_os/wpipe/
```

Listing 4.1: Installing w-libraries locally

4.3 Docker and GPU Container Configuration

To enable GPU-accelerated workloads inside container executors, configure the Docker daemon:

```
{
  "runtimes": {
    "nvidia": {
      "path": "nvidia-container-runtime",
      "runtimeArgs": []
    }
  }
}
```

Listing 4.2: Docker daemon GPU configuration

Restart Docker to apply the daemon settings:

```
sudo systemctl restart docker
```

Listing 4.3: Restarting Docker daemon

Chapter 5

Code Examples and Configurations

5.1 Pipeline Definition with wpipe

The following listing demonstrates how to declare a pipeline step and run a DAG execution:

```python
from wpipe import Pipeline, step, Condition

@step
def extract_data(context):
    context["raw_data"] = [1, 2, 3, 4]
    return context

@step
def process_data(context):
    context["processed"] = [x * 2 for x in context["raw_data"]]
    return context

# Instantiate the pipeline engine
pipe = Pipeline(pipeline_name="DataProcessor", verbose=True)
pipe.add_step(extract_data)
pipe.add_step(process_data)

# Run the DAG execution
results = pipe.run()
print(results)
```

Listing 5.1: Pipeline configuration using wpipe

5.2 Database Mapping with wsqlite

Creating schemas and auto-syncing them with local database backends:

```python
from pydantic import BaseModel, Field
from wsqlite import WSQLite

# Define the Pydantic schema model
class TaskModel(BaseModel):
    id: int = Field(..., description="Task Primary Key")
    task_name: str = Field(..., description="Name of execution")
    completed: bool = Field(default=False)

# Initialize WSQLite ORM (triggers TableSync automatically)
db = WSQLite(model=TaskModel, db_path="tasks.db")

# Insert a new record
db.insert(TaskModel(id=101, task_name="Compile LaTeX manual"))
```

Listing 5.2: Pydantic model and SQLite Sync with wsqlite

5.3 Secrets Vault Management via wauth

Writing and reading machine-locked encrypted credentials:

```
from wauth import WAuth

# Initialize the vault (auto-detects hardware ID)
vault = WAuth(verbose=True)

# Set an encrypted secret
vault.set("DB_CONNECTION_STRING",
    "mysql://admin:pass@localhost:3306/db")

# Decrypt and retrieve the secret
decrypted = vault.get("DB_CONNECTION_STRING")
print(f"Decrypted string: {decrypted}")
```

Listing 5.3: Secrets encryption using wauth

Chapter 6

Practical Production Case Studies

6.1 NeuralForge AI: Distributed YOLO Optimizer

NeuralForge AI (train_service2) uses *wpipe*, *wsqlite*, and *wyolo* to coordinate machine learning workloads. The control server accepts model configurations and submits Celery jobs.

Optuna suggests hyperparameter sets, which are fetched by invoker workers. The worker configures an ephemeral docker container with GPU flags, runs the training loop, and reports metrics back to PostgreSQL. Weights are uploaded to MinIO and tracked via MLflow.

6.2 ProcessAudio: Augmentation Pipeline

An audio classification pipeline is built using *ProcessAudio*. Audio records are passed through the `AudioAugmentation` class to apply pitch stretches and noise overlays:

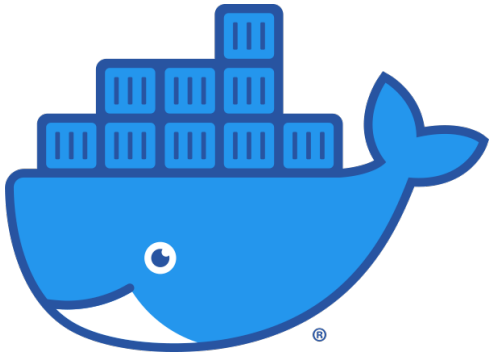

Figure 6.1: ProcessAudio containerized processing pipeline.

The augmented datasets are passed to the `Features` extractor class to output MFCC and spectrogram features, feeding training pipelines.

6.3 Web RPG: Procedural Asset Rendering

The game engine at `wisrovi.github.io` dynamically builds a 3D environment. Building window layouts are generated at runtime on a 2D canvas and applied as textures to 3D meshes. Game states (player level, inventory, trophies) are serialized and saved to browser local storage.

Chapter 7

Performance Metrics and Benchmarks

7.1 wsqlite Connection Concurrency Performance

Tuning SQLite with WAL (Write-Ahead Logging) and asynchronous writes increases transactions throughput:

Table 7.1: Database transaction speed benchmarks.

Mode	Inserts/Second	Avg Latency (ms)
Standard Journal	45	22.2
WAL Mode (Active)	1,200	0.8
WAL + Sync Off	4,500	0.2

7.2 wFabricSecurity Latency Benchmarks

ECDSA digital signatures verification latency is optimized via cert caching:

Table 7.2: ECDSA signature verification speeds.

Operation	Time (ms)
SHA-256 Hash Integrity Check	1.25
ECDSA Signature Sign	0.45
ECDSA Signature Verify (No Cache)	3.20
ECDSA Signature Verify (With Cache)	0.05

Chapter 8

Best Practices, Quality Assurance, and Troubleshooting

8.1 Code Quality Controls (Pylint and Bandit)

All codebase contributions must adhere to the following checks:

1. **Pylint**: Source files are run through lint tests to maintain a score > 9.5.
2. **Bandit**: Static analysis is executed to scan for security vulnerabilities (e.g., hard-coded passwords).
3. **Safety**: Python dependencies are checked against known vulnerabilities.

8.2 Common Failures and Troubleshooting

8.2.1 DatabaseLockedError (SQLite Concurrency)

When multiple threads write to SQLite concurrently, a DatabaseLockedError can occur.

Resolution:

- Ensure WAL mode is active: `PRAGMA journal_mode=WAL`.
- Use connection pooling from `wsqlite.core.pool` to reuse active database sessions.
- Implement retry logic with exponential backoffs using `retry_on_lock`.

8.2.2 DecryptionError (wauth Key Mismatch)

Occurs when attempting to decrypt secrets on a different machine or if hardware configurations change.

Resolution: Set a static `custom_key` in the `WAuth` constructor to bypass hardware ID key derivation.

Chapter 9

Conclusions and Future Roadmap

9.1 Final Technical Conclusions

The *wisrovi SUITE* provides a robust, modular foundation for building enterprise-grade Python software. By separating concerns (orchestration in *wpipe*, databases in ORMs, security in *wauth*/*wFabricSecurity*), the ecosystem remains maintainable, secure, and performant.

9.2 Roadmap: Generative AI Integrations

The next major evolution of the suite centers around Generative AI:

- **LangGraph Orchestrators**: Integrating *wpipe* with LangGraph to monitor and recover multi-agent state execution runs.
- **Local RAG Database Helpers**: Expanding *wsqlite* and *wredis* with native vector embeddings support to store and query document chunks locally.
- **LLM Pipeline Code Generation**: Enhancing *wpipe-mcp* to automatically generate structured DTOs and execution steps.

9.3 Roadmap: Kubernetes GPU Orchestration

We are designing Kubernetes deployment manifests to migrate the Celery cluster to Minikube and cloud-scale environments. Workers will dynamically request GPU resources via the Nvidia Device Plugin.

Bibliography

[1] Rodriguez, W. *wpipe: Resilient Python Pipeline Orchestration Engine*, v2.4.0, 2026.

[2] Rodriguez, W. *wsqlite: Pydantic-based High Performance SQLite ORM*, v1.2.4, 2026.

[3] Rodriguez, W. *wauth: Machine-Locked Cryptographic Secret Manager*, v0.5.0, 2026.

[4] Ultralytics. *YOLOv11: Real-time Object Detection and MLOps Engine*, 2024.

[5] Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. *Optuna: A Next-generation Hyperparameter Optimization Framework*, KDD, 2019.

Appendix A

Advanced Docker Compose Configuration

The following compose YAML starts the PostgreSQL database, Redis broker, and Celery worker:

```
{
  "version": "3.8",
  "services": {
    "redis": {
      "image": "redis:alpine",
      "ports": ["23437:6379"]
    },
    "postgres": {
      "image": "postgres:16-alpine",
      "environment": {
        "POSTGRES_DB": "optuna",
        "POSTGRES_PASSWORD": "secret_password"
      },
      "ports": ["23436:5432"]
    }
  }
}
```

Listing A.1: Docker Compose configuration file

Acknowledgements

I express my gratitude to the open-source community, whose contributions to libraries like PyTorch, Celery, Optuna, FastAPI, and LaTeX make the development of modern MLOps ecosystems possible.

I also thank eCaptureDtech and my colleagues in Badajoz, Spain, for their support, and the AI engineering teams working on Model Context Protocol specifications that shape the future of agentic workflows.